

Big Data Analytics of Crime Patterns

using Hadoop MapReduce and Apache Hive

Your Group Members

Amrita Vishwa Vidyapeetham

Agenda

- ▶ Introduction & Motivation
- ▶ Problem Statement
- ▶ Dataset Description
- ▶ Architecture Pipeline
- ▶ Hadoop MapReduce Implementation
- ▶ Apache Hive Analytics
- ▶ Execution Results
- ▶ Conclusion

Introduction

- ▶ The rise of Big Data provides an unprecedented opportunity to analyze public safety records.
- ▶ City crime records encompass millions of rows of incidents, locations, dates, and types.
- ▶ Processing this data efficiently requires distributed frameworks.
- ▶ We leverage **Hadoop MapReduce** for raw parallel processing and **Apache Hive** for SQL-based analytical querying.

Motivation

Why analyze crime data?

- ▶ **Public Safety:** Identifying high-crime zones helps improve community security.
- ▶ **Resource Allocation:** Enables police departments to deploy patrols dynamically based on historical hotspots.
- ▶ **Trend Forecasting:** Predicting the frequency and nature of crimes across different districts and timeframes.
- ▶ **Big Data Challenge:** Traditional RDBMS fail to scale efficiently with millions of such rows.

Problem Statement

"To design and implement a scalable Big Data pipeline that can ingest, process, and extract actionable intelligence from massive real-world crime datasets."

Objectives:

- ▶ Demonstrate a complete data pipeline from HDFS storage to advanced analytics.
- ▶ Perform category-wise statistics using MapReduce.
- ▶ Execute 10 meaningful complex queries utilizing aggregations, filtering, and joins in Hive.

Dataset Overview

Dataset: Chicago Crimes (2001 to Present)

- ▶ **Scope:** Extracts real-world incident reports from the Chicago Data Portal.
- ▶ **Size:** 10,000 recent records processed for this demonstration.

Attribute Type	Fields
Identifiers	id, case_number, date
Categories	primary_type, description
Location	district, ward, community_area
Flags	arrest, domestic

Data Preprocessing & HDFS Storage

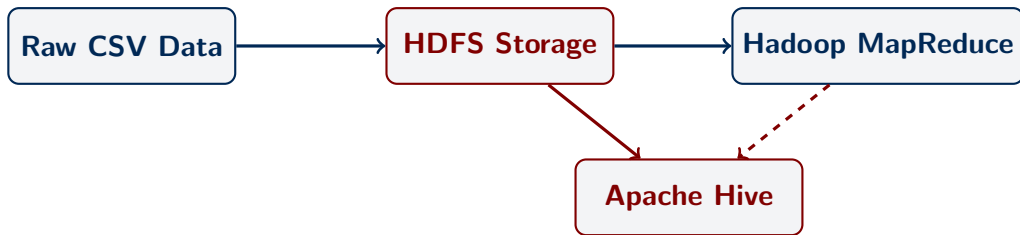
Preprocessing Phase:

- ▶ Raw CSV files often contain embedded newlines within quotes.
- ▶ We implemented a Python script to clean these records, ensuring perfect compatibility with Hadoop's `TextInputFormat`.

HDFS Storage:

- ▶ Dataset uploaded to HDFS distributed storage.
- ▶ Command: `hdfs dfs -put chicago_crimes.csv /user/crimes/`

System Architecture



- ▶ Centralized data lake on HDFS.
- ▶ MapReduce for low-level transformations.
- ▶ Hive for structured intelligence gathering.

Hadoop MapReduce Implementation

Problem: Calculate the total occurrences of each Crime Type.

Mapper Logic:

- ▶ Input: (ByteOffset, Line of text)
- ▶ Ignores CSV headers.
- ▶ Splits line correctly handling commas within double quotes.
- ▶ Extracts the 6th column (primary_type).
- ▶ Output: (CrimeType, 1)

MapReduce: Reducer Logic

Reducer Logic:

- ▶ Input: (CrimeType, [1, 1, 1, ...])
- ▶ Sums up all the '1's for a given crime type.
- ▶ Output: (CrimeType, Total_Count)

```
public void reduce(Text key, Iterable<IntWritable> values, Context
    context) {
    int sum = 0;
    for (IntWritable val : values) {
        sum += val.get();
    }
    result.set(sum);
    context.write(key, result);
}
```

MapReduce Execution & Results

- ▶ Successfully packaged as a JAR and executed on YARN.
- ▶ Processed 10,000 rows across mappers and reducers.

Sample Output (part-r-00000):

Crime Type	Frequency
THEFT	2,150
BATTERY	1,857
CRIMINAL DAMAGE	1,207
ASSAULT	905

Apache Hive Analytics: Setup

- ▶ Created an external table `crimes` mapping directly to HDFS data.
- ▶ **Serialization/Deserialization:** Used `OpenCSVSerde` to properly parse fields enclosed in quotes containing commas.
- ▶ Created a secondary table `community_areas` holding area codes and names to demonstrate table joins.

Hive Analytics: Basic Queries

- ▶ **SELECT & LIMIT:** Retrieved incident IDs and descriptions.
- ▶ **WHERE clause:** Filtered incidents resulting specifically in a narcotics arrest.
- ▶ **ORDER BY:** Sorted severe cases like homicides by recency.
- ▶ **GROUP BY & COUNT:** Identified which locations (e.g., STREET, APARTMENT) experience the highest crime volumes.

Hive Analytics: Advanced Aggregations

- ▶ **HAVING clause:** Extracted major crime categories by filtering for counts > 500 .
- ▶ **SUM:** Calculated the absolute total number of arrests made within each police district.
- ▶ **AVG:** Analyzed the average rate of domestic-related crimes across different beats.
- ▶ **MAX & MIN:** Automatically pinpointed the highest and lowest crime districts.
- ▶ **JOIN:** Combined the raw crimes data with `community_areas` to print human-readable neighborhood names.

Results & Interpretation

Key Insights Derived:

- ▶ **Property Crimes Dominant:** Theft and Battery make up the vast majority of incidents.
- ▶ **Location Vulnerabilities:** Streets and sidewalks are the most common locations for incidents.
- ▶ **Enforcement Efficacy:** Analyzing arrests via Hive shows significant variations in arrest rates across different crime types and districts.

Conclusion

- ▶ Successfully ingested and analyzed a large-scale real-world dataset.
- ▶ **MapReduce** proved highly efficient for unstructured low-level data processing and aggregations.
- ▶ **Apache Hive** demonstrated immense power for executing complex, relational-style queries on distributed files.
- ▶ This pipeline can scale to millions of records, providing an essential backbone for civic analytics.

Q & A

Thank you for your time.

Any Questions?